OMIS 105: Introduction to Database Management Systems

Week 1 — Foundations

Instructor: Dr. Parsian

Course Overview

- **Course**: OMIS 105 — Introduction to Database Management Systems
- **Prerequisite**: OMIS 30 (Introduction to Programming)
- **Duration**: 10 weeks, 2 sessions × 2 hours per week
- **Tools**: DuckDB, SQL, Marimo notebooks, qStudio
- **Domain**: E-commerce database (ShopSmart Inc.)

What You Will Learn

1. How databases store and organize data
2. The relational model and SQL
3. Database design and normalization
4. Joins and subqueries across many tables
5. Ranking functions (top N, ties, second-highest)
6. Transactions and data integrity

Week-by-Week Roadmap

Week	Topic
1	Database Foundations
2	Relational Modeling (keys, relationships)
3	SQL Basics (CASE, string functions)
4	Aggregation (GROUP BY, HAVING) and first JOINs
5	SQL Joins
6	Joins II (LEFT/RIGHT JOIN, self-joins)
7	Subqueries
8	Database Design & Normalization
9	Ranking Functions (ROW_NUMBER, RANK, DENSE_RANK)

Session 1: Why Databases?

The Data Problem

Imagine you run **ShopSmart**, an online store:

- 64 products across 8 categories
- 40 customers
- 200 orders with about 600 line items (order lines)

How do you store and manage all this data?

Option 1: Flat Files (Spreadsheets)

```
product_id,product_name,category,price,stock_quantity
1,Smartphone X12,Electronics,321.52,12
2,Laptop Pro 15,Electronics,372.07,125
...
```

(These are the first lines of this week's `data/products.csv`.)

Seems simple enough… right?

Problems with Flat Files

- **Redundancy**: Category "Electronics" repeated for every electronics product
- **Inconsistency**: What if someone types "Electronicss"?
- **No concurrent access**: Two employees editing the same file?
- **No security**: Everyone sees everything
- **Scale**: Excel stops at about 1 million rows (1,048,576) per sheet

Option 2: A Database

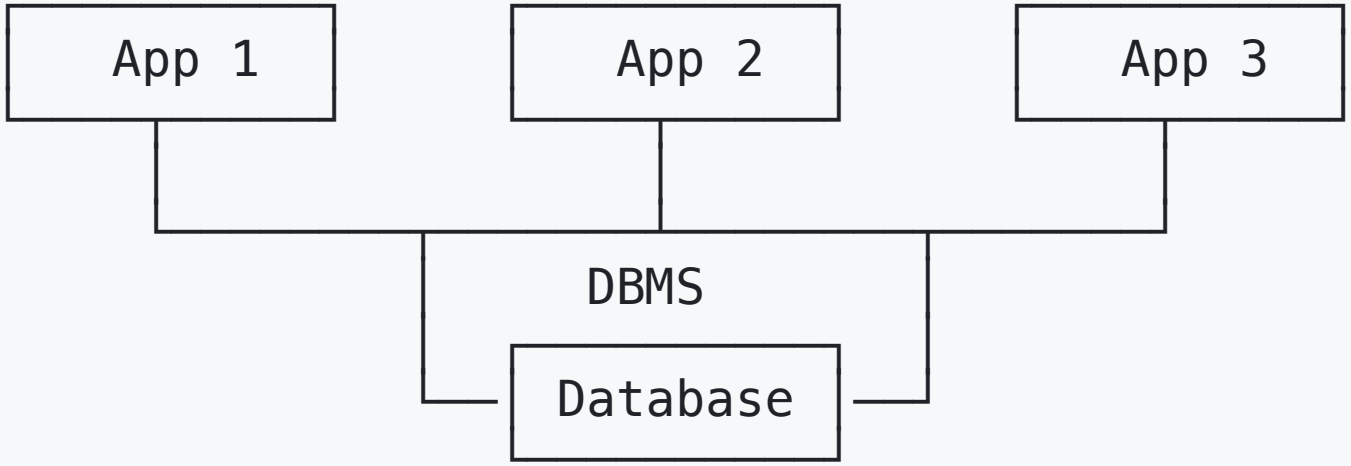
A **database** is an organized collection of structured data, stored electronically and managed by a **Database Management System (DBMS)**.

A DBMS provides:

- Structured storage
- Query language (SQL)
- Concurrent access
- Security and access control
- Data integrity enforcement

What Is a DBMS?

Database Management System — software that sits between applications and data.



Popular DBMS Software

DBMS	Type	Use Case
Oracle	Enterprise Relational	Banking, large corps
PostgreSQL	Open-source Relational	Web apps, analytics
MySQL	Open-source Relational	Web applications
SQL Server	Enterprise Relational	Microsoft ecosystem
MongoDB	Document (NoSQL)	Flexible schemas
DuckDB	Analytical Relational (embedded)	Data analysis, education

Why DuckDB for This Course?

- **Zero setup**: No server needed — runs inside your program (in-process)
- **Standard SQL**: Standard SQL, plus some friendly extras
- **CSV-friendly**: Load data from CSV files directly
- **Fast**: Columnar storage, vectorized execution
- **Python integration**: Works great in Marimo/Jupyter notebooks
- **Free & open source**

Key Database Concepts

- Table
- Row
- Column
- Primary Key
- Foreign Key

Tables

A **table** is a collection of **related data** organized in rows and columns.

- Table name: `products`
- Column names:
  - `product_id`
  - `product_name`
  - `category`
  - `price`

product_id	product_name	category	price
1	Smartphone X12	Electronics	321.52
2	Laptop Pro 15	Electronics	372.07

Rows and Columns

- **Row** (record/tuple): A single data entry
  - Example: All info about "Smartphone X12"
- **Column** (field/attribute): A single property
  - Example: All product prices
- **Cell**: The intersection of a row and column
  - Example: The price of Smartphone X12 = 321.52

Schema

A **schema** defines the structure of a database:

- What tables exist
- What columns each table has
- What data types each column holds
- What constraints apply

```
CREATE TABLE products (
    product_id    INTEGER PRIMARY KEY,
    product_name  VARCHAR,
    category      VARCHAR,
    price         DECIMAL(10,2),
    stock_quantity INTEGER
);
```

Data Types

Type	Description	Example
INTEGER	Whole numbers	`42`
DECIMAL(p,s)	Exact decimals	`29.99`
VARCHAR	Variable-length text	`'Laptop Pro'`
DATE	Calendar date	`DATE '2024-06-15'`
BOOLEAN	True/False	`TRUE`
TIMESTAMP	Date and time	`TIMESTAMP '2024-06-15 14:30:00'`

`DECIMAL(10,2)` = up to 10 digits in total, 2 of them after the decimal point.

Constraints

Rules that enforce data integrity:

Constraint	Purpose
PRIMARY KEY (PK)	Uniquely identifies each row
NOT NULL	Column must have a value (cannot be `NULL`)
UNIQUE	No duplicate values allowed
CHECK	Custom validation rule
DEFAULT	Auto-fill value if none given
FOREIGN KEY (FK)	Links to another table (Week 2)

Example with Constraints

```
CREATE TABLE products (
    product_id     INTEGER PRIMARY KEY,
    product_name   VARCHAR NOT NULL,
    category       VARCHAR NOT NULL,
    price          DECIMAL(10,2) CHECK (price > 0),
    stock_quantity INTEGER DEFAULT 0 CHECK (stock_quantity >= 0)
);
```

Now DuckDB rejects a product with a negative price, or with no name.

Session 2: Hands-On with DuckDB

Installing DuckDB

Python (pip):

```
pip install "duckdb>=2.0" marimo
```

(Full steps for Mac and Windows: the `software_installation/` folder.)

In a Marimo/Jupyter Notebook:

```
import duckdb

# Create an in-memory database
con = duckdb.connect()
print(con.sql("SELECT 'Hello, DuckDB!' AS greeting"))
```

Your First Query

```
import duckdb
con = duckdb.connect()

result = con.sql("SELECT 42 AS answer")
print(result)
```

Output:

```
┌─────────┐
│ answer  │
│  int32  │
├─────────┤
│      42 │
└─────────┘
```

Loading CSV Data

```
import duckdb
con = duckdb.connect()

# Load products.csv directly (run from the week01 folder)
con.sql("""
    CREATE OR REPLACE TABLE products AS
    SELECT * FROM read_csv('./data/products.csv')
""")

# See what we loaded
con.sql("SELECT * FROM products LIMIT 5").show()
```

`read_csv` detects the column names and data types for you.

`CREATE OR REPLACE` lets you run the cell again without an error.

Examining Table Structure

```
-- Show all tables
SHOW TABLES;

-- Describe a table's columns
DESCRIBE products;

-- Count rows
SELECT COUNT(*) AS total_products
FROM products;                          -- 64
```

Basic SELECT

```sql
-- All columns, all rows
SELECT *
FROM products;

-- Specific columns
SELECT product_name, price
FROM products;

-- With a condition
SELECT product_name, price
FROM products
WHERE price > 100;                    -- 28 products
```

Anatomy of a SELECT Statement

```sql
SELECT   column1, column2     -- What to show
FROM     table_name           -- Where to look
WHERE    condition            -- Which rows
ORDER BY column1              -- Sort results
LIMIT    10;                  -- How many rows
```

Each clause has a purpose, and they must appear in this order.

We will practice them in depth in Week 3.

Filtering with WHERE

```sql
-- Exact match
SELECT * FROM products WHERE category = 'Electronics';

-- Numeric comparison
SELECT * FROM products WHERE price < 50;          -- 18 products

-- Combining conditions
SELECT * FROM products
WHERE category = 'Books' AND price < 30;          -- Python Crash Course (8.85)
```

Text values use **single** quotes, and are case-sensitive: `'books'` ≠ `'Books'`.

Sorting with ORDER BY

```sql
-- Ascending (default)
SELECT product_name, price
FROM products
ORDER BY price;

-- Descending
SELECT product_name, price
FROM products
ORDER BY price DESC;

-- Multiple columns
SELECT product_name, category, price
FROM products
ORDER BY category, price DESC;
```

Limiting Results

```sql
-- Top 5 most expensive products
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 5;

-- Skip first 10, then get 5 (rows 11–15)
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 5 OFFSET 10;
```

Always use `ORDER BY` with `LIMIT` — otherwise "the top 5" can be any 5 rows.

Aliases

```sql
-- Column alias
SELECT product_name AS name,
       price AS unit_price
FROM products;

-- Computed column with alias (8.75% sales tax)
SELECT product_name,
       ROUND(price * 1.0875, 2) AS price_with_tax
FROM products;
```

DISTINCT Values

```sql
-- All unique categories
SELECT DISTINCT category
FROM products;

-- Count of unique categories
SELECT COUNT(DISTINCT category) AS num_categories
FROM products;                          -- 8
```

NULL Values

`NULL` means "unknown" or "missing" — not zero, not empty string.

```sql
-- Check for NULL
SELECT *
FROM products
WHERE stock_quantity IS NULL;

-- Check for NOT NULL
SELECT *
FROM products
WHERE stock_quantity IS NOT NULL;

-- CAUTION: this does NOT work — it always returns 0 rows
-- SELECT * FROM products WHERE stock_quantity = NULL;
```

In our `products.csv`, every product has a stock quantity, so `IS NULL` returns 0 rows.

The LIKE Operator

Pattern matching for text:

```sql
-- Starts with 'S'
SELECT * FROM products WHERE product_name LIKE 'S%';

-- Contains 'Pro'
SELECT * FROM products WHERE product_name LIKE '%Pro%';

-- Starts with 'B' and is exactly 5 characters long
SELECT DISTINCT category FROM products WHERE category LIKE 'B____';
```

`%` = any sequence of characters (including none)

`_` = exactly one character

- `'S%'` → Smartphone X12, SQL Cookbook, Silk Scarf, Shower Curtain, Stuffed Bear, Science Set

The IN Operator

```sql
-- Instead of multiple ORs
SELECT * FROM products
WHERE category IN ('Electronics', 'Books', 'Sports');

-- Equivalent but longer:
SELECT * FROM products
WHERE category = 'Electronics'
   OR category = 'Books'
   OR category = 'Sports';
```

BETWEEN Operator

```sql
-- Price range (inclusive: 20 and 100 are included)
SELECT product_name, price
FROM products
WHERE price BETWEEN 20 AND 100;     -- 30 products

-- Equivalent to:
SELECT product_name, price
FROM products
WHERE price >= 20 AND price <= 100;
```

Basic Aggregate Functions

```sql
-- Count all products
SELECT COUNT(*) AS total FROM products;

-- Average price
SELECT AVG(price) AS avg_price FROM products;

-- Min and Max
SELECT MIN(price) AS cheapest,
       MAX(price) AS most_expensive
FROM products;

-- Sum of stock
SELECT SUM(stock_quantity) AS total_stock
FROM products;
```

An aggregate turns **many rows into one value**.

Combining Aggregates

```sql
SELECT
    COUNT(*) AS total_products,
    ROUND(AVG(price), 2) AS avg_price,
    MIN(price) AS min_price,
    MAX(price) AS max_price,
    SUM(stock_quantity) AS total_inventory
FROM products;
```

total_products	avg_price	min_price	max_price	total_inventory
64	98.27	8.85	446.63	13037

DuckDB Special Features

```sql
-- Read CSV without creating a table
SELECT * FROM read_csv('./data/products.csv') LIMIT 5;

-- Export query results to CSV
-- (this is how data/expensive_products.csv was made: 28 rows)
COPY (
    SELECT product_name, category, price
    FROM products
    WHERE price > 100
    ORDER BY price DESC
) TO './data/expensive_products.csv' (HEADER, DELIMITER ',');

-- Get column statistics (min, max, average, NULL count, ...)
SUMMARIZE products;
```

File System as Database

DuckDB can query files directly:

```
-- Query a CSV file as if it were a table
SELECT category, COUNT(*) AS cnt
FROM read_csv('./data/products.csv')
GROUP BY category
ORDER BY cnt DESC;
```

No `CREATE TABLE` needed! (Each category has 8 products. `GROUP BY` is a preview — it comes in Week 3.)

Saving Your Work

```
# In-memory (default) — lost when you close
con = duckdb.connect()

# Persistent — saved to file
con = duckdb.connect('shopsmart.duckdb')

# Now all tables persist between sessions
```

A `.duckdb` file holds the whole database: every table, in one file.

Summary: Key Terms

Term	Definition
Database	Organized collection of structured data
DBMS	Software to manage databases
Table	Data organized in rows and columns
Row	Single record in a table
Column	Single attribute/field
Schema	Structure definition of a database
SQL	Language to interact with databases
Query	A request for data from a database

Summary: SQL So Far

```
SELECT columns FROM table
WHERE conditions
ORDER BY columns [ASC|DESC]
LIMIT n;
```

Key operators: `=`, `<>`, `<`, `>`, `LIKE`, `IN`, `BETWEEN`, `IS NULL`

Aggregate functions: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`

What Is Next?

Week 2: Relational Thinking

- Primary and foreign keys
- Relationships between tables
- Entity-Relationship diagrams
- Multiple related tables

Practice Makes Perfect

- Complete **Lab 1** (loading data, basic queries)
- Explore the `./data/products.csv` dataset
- Try writing your own queries
- Install DuckDB and Marimo, and experiment!

Questions?

Thank you!

OMIS 105 — Introduction to Database Management Systems — Fall 2026